

Session 1 · Deck 1.B1

The Trunk: From the Planner to Equilibrium

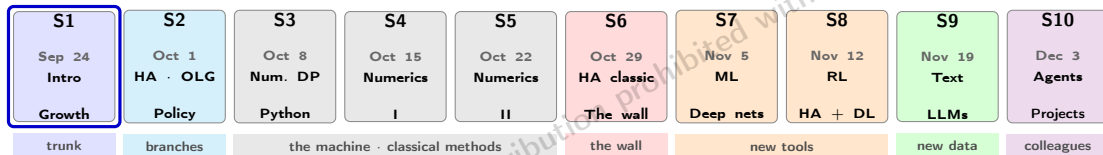
Zhigang Feng

Thursday, September 24, 2026

Build it on paper → solve it on the machine → push it to the frontier with AI

Where We Are

▼ you are here: 1.B1

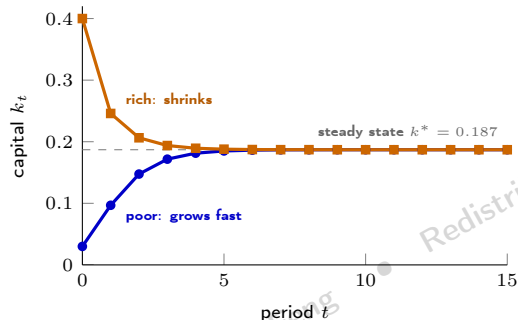


Build it on paper → solve it on the machine → push it to the frontier with AI

Block B: 1.B1 the optimal growth model and its equilibrium · 1.B2 why dynamic programming works and how it computes · 1.B3 from dynamic programming to reinforcement learning, and from one agent to many.

One Model, Three Roads to the Same Answer

Two economies, same preferences and technology



$u = \ln c$, $y = k^{0.36}$ full depreciation, $\beta = 0.95$: exact policy

$$k_{t+1} = \alpha \beta k_t^\alpha$$

The planner: maximize welfare subject to the resource constraint.

The market: households and firms optimize taking prices as given; prices clear markets.

The computer: iterate a Bellman operator until it stops changing.

Here all three roads meet. The rest of the course is about what happens when they *don't*: incomplete markets (no planner), taxes and frictions (market $\neq$ planner), and too many dimensions (the computer cannot iterate).

The Environment

- **Time** $t = 0, 1, 2, \dots$; a unit mass of identical, infinitely lived households.
- **Preferences:** $\sum_{t=0}^{\infty} \beta^t u(c_t)$ with $\beta \in (0, 1)$; $u' > 0$, $u'' < 0$, $\lim_{c \rightarrow 0} u'(c) = \infty$.
- **Technology:** $Y_t = z_t F(K_t, N_t)$ with constant returns; per worker $f(k) = F(k, 1)$, $f' > 0$, $f'' < 0$, Inada conditions. Labor is supplied inelastically, $N_t = 1$.
- **Resource constraint:**

$$c_t + k_{t+1} = z_t f(k_t) + (1 - \delta)k_t, \quad k_0 \text{ given.}$$

- **Shocks:** z_t follows a Markov process (next slides).

From Solow to Ramsey–Cass–Koopmans

Solow fixes the saving rate s . Here saving is **chosen**: households trade consumption today against capital tomorrow. That single change turns an accounting model into an *optimization* model — and brings in expectations, prices, and welfare.

Everything later in the course is this environment with **more states** (assets, income, age, health), **more agents**, **frictions**, or a **government**.

The Sequence Problem: Euler Equation and Transversality

Deterministic case ($z_t \equiv 1$). The Lagrangian

$$\mathcal{L} = \sum_{t=0}^{\infty} \beta^t \left\{ u(c_t) + \lambda_t [f(k_t) + (1 - \delta)k_t - c_t - k_{t+1}] \right\}$$

gives $u'(c_t) = \lambda_t$ and $\lambda_t = \beta \lambda_{t+1} [f'(k_{t+1}) + 1 - \delta]$, i.e.

$$u'(c_t) = \beta u'(c_{t+1}) [f'(k_{t+1}) + 1 - \delta]$$

plus the **transversality condition** $\lim_{t \rightarrow \infty} \beta^t u'(c_t) k_{t+1} = 0$.

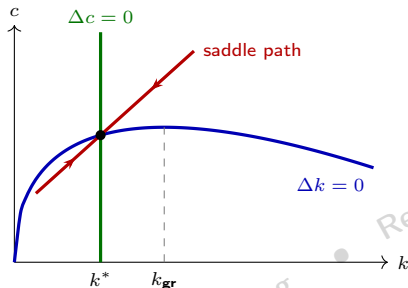
Reading the Euler equation

Give up one unit of consumption today (cost $u'(c_t)$); invest it; tomorrow it returns $f'(k_{t+1}) + 1 - \delta$ units, worth $\beta u'(c_{t+1})$ each. At an optimum the two sides balance.

Necessary, not sufficient

Many paths satisfy the Euler equation — most of them explode. With concave u and f , **Euler + transversality** is sufficient for an optimum (Stokey–Lucas–Prescott, Thm. 4.15). Remember this when we train neural networks on Euler residuals (S7).

Steady State and Transitional Dynamics



Continuous-time intuition; discrete time is analogous.

- **Steady state:** $c_t = c_{t+1}$ in the Euler equation gives the *modified golden rule*

$$f'(k^*) = \frac{1}{\beta} - 1 + \delta > \delta = f'(k_{gr}),$$

so impatience keeps k^* below the golden rule.

- **Dynamics:** the linearized system has one stable and one unstable root. From any k_0 there is a **unique** c_0 on the saddle path; every other choice violates feasibility or transversality.
- **Economics:** convergence — capital-poor economies grow faster.

Counting stable roots to pin down the jump variable is the **Blanchard–Kahn** logic we use for perturbation methods (S5).

Adding Uncertainty: Markov Shocks

- **Finite Markov chain:** $z_t \in \{z_1, \dots, z_m\}$ with $\Pi_{ij} = \Pr(z_{t+1} = z_j \mid z_t = z_i)$. Tomorrow's distribution depends only on today's state.
- **Stationary distribution:** $\pi^\top = \pi^\top \Pi$.
- **Example:** $\Pi = \begin{pmatrix} 0.9 & 0.1 \\ 0.2 & 0.8 \end{pmatrix}$ gives $\pi = (\frac{2}{3}, \frac{1}{3})$: from $\pi_L = 0.9\pi_L + 0.2\pi_H$.
- A continuous AR(1), $\ln z' = \rho \ln z + \sigma \varepsilon'$, is approximated by such a chain (Tauchen, Rouwenhorst: S4).

Stochastic Euler equation

$$u'(c_t) = \beta \mathbb{E}_t \left[u'(c_{t+1}) (z_{t+1} f'(k_{t+1}) + 1 - \delta) \right]$$

Why Markov matters. Because (k_t, z_t) contains everything needed to forecast the future, the problem looks the same every period from the same state — that is what makes a *recursive* formulation possible.

A distribution already appears. Shocks make k_t wander: the economy has a *stationary distribution of capital over time*. In S2 the same object reappears as a distribution of wealth *across people*.

A Benchmark with a Closed Form (Brock–Mirman, 1972)

Take $u(c) = \ln c$, $y = z k^\alpha$, and full depreciation $\delta = 1$.

Guess $V(k, z) = A + B \ln k + D \ln z$ and verify:

$$k' = \alpha\beta z k^\alpha, \quad c = (1 - \alpha\beta) z k^\alpha, \quad B = \frac{\alpha}{1 - \alpha\beta}.$$

- Households save a **constant share** $\alpha\beta$ of output (= 0.342 for $\alpha = 0.36$, $\beta = 0.95$).
- $\ln k_{t+1} = \ln(\alpha\beta) + \alpha \ln k_t + \ln z_t$: log capital is autoregressive, with a stationary distribution.

Homework 1 begins exactly here.

Why a toy model matters

Closed forms are rare — which is exactly why this one is precious. **Every numerical method in this course is first tested here:** value function iteration (S3), perturbation and projection (S5), neural networks (S7), reinforcement learning (S8). If a method cannot recover $k' = \alpha\beta z k^\alpha$, do not trust it on anything harder.

Decentralization: Competitive Equilibrium

Households own capital, rent it and their labor to firms:

$$\max \sum_t \beta^t u(c_t) \quad \text{s.t.} \quad c_t + k_{t+1} = (1 + r_t - \delta) k_t + w_t.$$

Firms rent inputs: $\max z_t F(K_t, N_t) - r_t K_t - w_t N_t$, so

$$r_t = z_t f'(K_t), \quad w_t = z_t [f(K_t) - K_t f'(K_t)].$$

Markets clear: $k_t = K_t$, $N_t = 1$, and the goods market clears.

Definition (sequence form)

A competitive equilibrium is a sequence of prices $\{r_t, w_t\}$ and allocations $\{c_t, k_{t+1}, K_t\}$ such that (i) households optimize given prices, (ii) firms optimize given prices, and (iii) all markets clear.

The household Euler equation $u'(c_t) = \beta u'(c_{t+1})(1 + r_{t+1} - \delta)$ with $r_{t+1} = f'(K_{t+1})$ is the planner's Euler equation. Same equations $\Rightarrow$ same allocation.

The Welfare Theorems

- **First welfare theorem:** every competitive equilibrium allocation is Pareto optimal.
- **Second welfare theorem:** every Pareto optimum can be supported as a competitive equilibrium (with transfers; needs convexity).
- **The shortcut:** compute the equilibrium by solving *one* planner's problem — no prices, no fixed point.

When the shortcut is gone

- **Uninsurable risk, borrowing limits** — incomplete markets (Aiyagari, S2).
- **Distortionary taxes** — wedges between private and social returns (optimal policy, S2).
- **Externalities, market power, nominal rigidities.**
- **Overlapping generations** — even competitive equilibria can be inefficient (S2).

Without the welfare theorems we must solve for **prices and allocations jointly** — equilibrium becomes a *fixed point*, and that is where most of the computational difficulty in modern macro comes from.

Recursive Competitive Equilibrium: “Little k , Big K ”

A household with capital k lives in an economy with aggregate capital K . Prices depend on the *aggregate* state; to plan, the household needs a **perceived law of motion** $K' = H(K, z)$:

$$v(k, K, z) = \max_{c, k'} \left\{ u(c) + \beta \mathbb{E}[v(k', K', z') \mid z] \right\}$$

s.t. $c + k' = (1 + r(K, z) - \delta)k + w(K, z)$, $K' = H(K, z)$.

Definition (RCE)

A value function v , policy $g(k, K, z)$, prices $r(K, z)$, $w(K, z)$, and a law of motion $H(K, z)$ such that:

1. v, g solve the household problem given r, w, H ;
2. prices equal marginal products;
3. **consistency**: $g(K, K, z) = H(K, z)$ for all (K, z) ;
4. the resource constraint holds.

Research in Practice

Consistency is a **fixed point in functions**: the forecasting rule people use must be the rule their own choices generate. Here the planner solves it for free; in heterogeneous-agent and optimal-policy models it is the *computational core*.

One Template for Every Model in This Course

Model	Individual state	Aggregate state	Equilibrium consistency	Where
Growth model (RCE)	k, z	K, z	individual policy at $k = K$ reproduces $H(K, z)$	S1
Aiyagari (stationary)	a, z	distribution λ (constant)	λ invariant under g ; $K = \int a d\lambda$ clears the capital market	S2, S6
Krusell–Smith	a, ε, z	distribution Γ, z	simulated Γ' matches the forecast rule $H(\Gamma, z, z')$	S2, S6, S8
Life-cycle OLG	age j, a, z	cohort distributions	cohort aggregates clear markets period by period	S2, S6
Taxation without commitment	a, z	distribution Γ	perceived future tax rule = actual best response	S2, S6, S8

Learn the RCE template once. Every model after today asks the same three questions: **what is the state, what must agents forecast, and what makes the forecast correct?**

Takeaways

1. The growth model has **three equivalent characterizations**: the sequence problem (Euler + transversality), the planner's problem, and the competitive equilibrium.
2. The steady state satisfies the **modified golden rule**; the saddle path gives a unique convergent solution.
3. Markov shocks make a **recursive** formulation possible — and already generate a stationary distribution.
4. The **Brock–Mirman** closed form is the benchmark for every method we build.
5. The **welfare theorems** let a planner stand in for markets; when they fail, equilibrium is a fixed point.
6. The **recursive competitive equilibrium** — individual state, aggregate state, law of motion, consistency — is the template for every model in the course.

Bridge: Why Should the Recursive Road Be Valid?

We wrote $v(k, K, z) = \max\{u(c) + \beta \mathbb{E} v(\cdot)\}$ as if replacing an infinite sequence by one function were obviously fine. It is not obvious:

- Does the Bellman equation give the *same* answer as the sequence problem?
- Does it have a solution — only one — and can a computer find it?
- Which properties of the solution (monotone, concave, smooth) can algorithms exploit?

Next (1.B2): the theory behind dynamic programming, and how that theory becomes an algorithm.